UNIVERSITÄT OSNABRÜCK

Programmieren für PsychologInnen

Schleifen in Python

Allgemeine Psychologie I
B.Sc. Psychologie / M.Sc. Psychologie - Seminar Programmieren
16.05.2025

Merle Sagehorn, M.Sc.

AvaVR – VR-EEG-Studie

- Im Rahmen dieser Studie werden Ihnen in VR menschliche Avatare gezeigt, während EEG gemessen wird.
- 2.5 VP Stunden oder 15€
- Grundvoraussetzungen:
  - keine Teilnahme an der vorangegangenen Pilotstudie (AvaVR - Pilot)
  - Alter: zwischen 18 und 39 Jahren
  - keine Sehschwäche (betrifft jegliche Dioptrienwerte, auch Lesebrillen; Teilnahme mit Kontaktlinsen möglich, mit Brille nicht)
  - sehr gute Deutschkenntnisse (mindestens B2-Niveau)
  - keine akute psychische und/oder neurologische Erkrankung (z.B. affektive Störungen, Epilepsie, Migräne)
  - keine akute psychiatrische und/oder psychotherapeutische Behandlung
  - keine Einnahme von zentralnervös-wirksamen Substanzen (z.B. Cannabis, Insulin, Psychopharmaka)
  - keine auswaschbare Haarfärbung (keine Tönung, keine Henna-Farben)
  - keine Dreadlocks

Today

- Fragen & Aufgaben If-Else
- Schleifen
  - For Loop
  - While Loop
- print() advanced
- Textfiles einlesen & schreiben

Boolean Variables / Logische Variablen

```
yn = True
ny = False

yn == ny #Kommt hier ein Output raus?
```

```
res = yn == ny
print(res)

# Output: False
```

- == ist eine relationale Operation
  - genauso wie <, >, !=, <=, >=
- Ergebnis kann in einer Variable gespeichert & als Output ausgegeben werden

Recap: If/Match-Statement

```python
# IF-STATEMENTS & MATCH-STATEMENTS

# Tageszeit bestimmen:
# Eingabe der Uhrzeit in Stunden vom Benutzer
hour = int(input("Bitte geben Sie die Stunde des Tages ein (0-23): "))

# Überprüfung der Eingabe und Ausgabe der entsprechenden Nachricht
if hour < 0 or hour > 23:
    print("Ungültige Eingabe. Bitte geben Sie eine Stunde zwischen 0 und 23 ein.")
elif hour < 12:
    print("Guten Morgen!")
elif hour < 18:
    print("Guten Tag!")
elif hour < 21:
    print("Guten Abend!")
else:
    print("Gute Nacht!")
```

Recap: If/Match-Statement

```python
uhrzeit = input("Wie spät ist es? (HH:MM): ")

#festlegen, dass Uhrzeit jetzt nur in HH:MM akzeotiert wird
try:
    stunde = int(uhrzeit.split(":")[0])
except (ValueError, IndexError):
    print("Ungültiges Format. Bitte im Format HH:MM eingeben.")
    exit()

#IF-Statements
if 5 <= stunde <= 11:
    print("Guten Morgen!")
elif 12 <= stunde <= 17:
    print("Guten Nachmittag!")
elif 18 <= stunde <= 21:
    print("Guten Abend!")
elif 22 <= stunde <= 23 or 0 <= stunde <= 4:
    print("Gute Nacht!")
else:
    print("Ungültige Stunde.")
```

uhrzeit = “20:47“

uhrzeit.split(“:“)
→ string anhand aller vorkommenden : splitten & als einzelne Elemente in eine Liste schreiben
Output: [20, 47]

uhrzeit.split(“:“).[0]
→ Eintrag an erster Stelle der Liste, hier: Stunde 20

Recap: If/Match-Statement

```python
# Wetterwarnung:
# Eingabe der Temperatur in Celsius vom Benutzer
temperature = float(input("Bitte geben Sie die Temperatur in Celsius ein: "))

# Überprüfung der Temperatur und Ausgabe der entsprechenden Nachricht
if temperature < 0:
    print("Es ist eisig.")
elif 0 <= temperature <= 13:
    print("Es ist kalt.")
elif 14 <= temperature <= 25:
    print("Es ist angenehm.")
else:
    print("Es ist heiß.")
```

Recap: If/Match-Statement

```python
### Benutzeranmeldung:
# Vorgedefinierte Anmeldedaten
correct_username = 'Benutzer'
correct_password = 'Passwort123'

# Benutzereingabe
username = str(input("Benutzername: "))
password = str(input("Passwort: "))

# Überprüfung der Anmeldedaten mit match
if username == correct_username and password == correct_password:
    print("Anmeldung erfolgreich!")
else:
    print("Benutzername oder Passwort ist falsch.")
```

Recap: If/Match-Statement

```
### Passwortstärke:
# Eingabe des Passworts vom Benutzer
password = input("Bitte geben Sie ein Passwort ein: ")

# Überprüfung der Kriterien
length_ok = len(password) >= 8
has_upper = password != password.lower()  # Überprüfen, ob das Passwort mindestens einen
Großbuchstaben hat

# Überprüfung der Passwortstärke mit match
match (length_ok, has_upper):
    case (True, True):
        print("Das Passwort ist stark.")
    case _:
        print("Das Passwort ist schwach. Bitte verwenden Sie mindestens 8 Zeichen und
mindestens einen Großbuchstaben.")
```

Recap: If/Match-Statement & Loops

```
### Zusatz: Prüft auch auf mindestens eine Zahl und ein Sonderzeichen
password = input("Bitte geben Sie ein Passwort ein: ")

# Variablen zur Überprüfung der Kriterien
has_upper = any(c.isupper() for c in password)  # mindestens ein Großbuchstabe
has_digit = any(c.isdigit() for c in password)  # mindestens eine Zahl
has_special = any(c in '!@#$%^&*(),.?":{}|<>' for c in password)  # mindestens ein
Sonderzeichen
length_ok = len(password) >= 8  # mindestens 8 Zeichen

# Überprüfung der Passwortstärke mit match
match (length_ok, has_upper, has_digit, has_special):
    case (True, True, True, True):
        print("Das Passwort ist stark.")
    case _:
        print("Das Passwort ist schwach. Bitte verwenden Sie mindestens 8 Zeichen,
einschließlich Großbuchstaben, Zahlen und Sonderzeichen.")
```

Loops!

Recap: If/Match-Statement

```python
# Aufgabe 4

import re  # Für reguläre Ausdrücke

passwort_check = input("Bitte geben Sie Ihr neu gewähltes Passwort ein: ")

# Bedingungen für ein starkes Passwort
zeichen = len(passwort_check) >= 8
grossbuchstabe = any(c.isupper() for c in passwort_check)
zahl = any(c.isdigit() for c in passwort_check)
sonderzeichen = re.search(r"[!@#$%^&*()_+{}\[\]:;<>,.?/~\\\-]", passwort_check)

if zeichen and grossbuchstabe:
    print("Das Passwort ist stark (Grundbedingungen erfüllt).")
    if zahl and sonderzeichen:
        print("Bonus: Passwort enthält auch Zahl und Sonderzeichen.")
else:
    print("Das Passwort ist schwach. Bitte wähle ein stärkeres.")
```

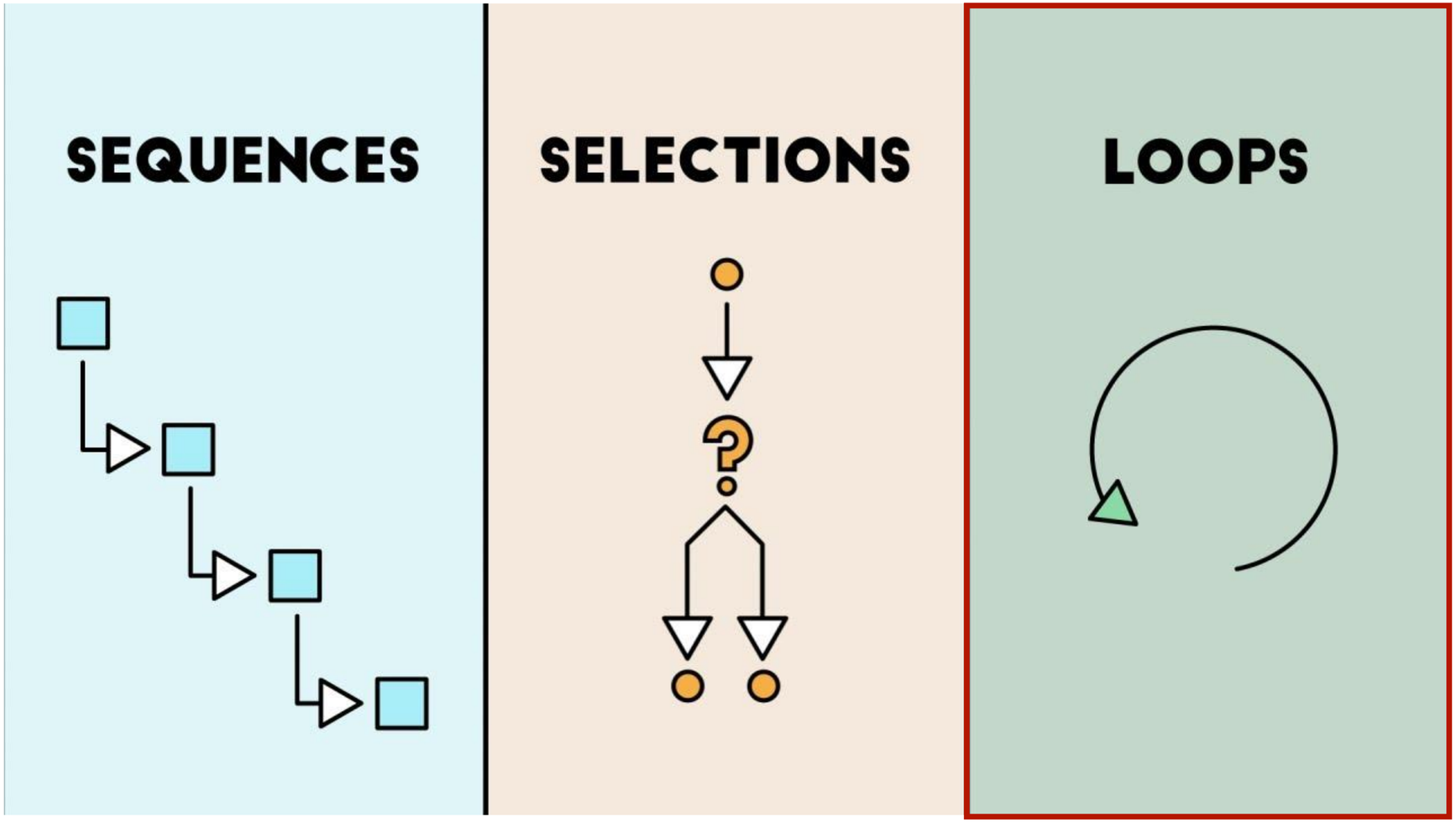
re.search(pattern, string) sucht nach einem bestimmten Pattern innerhalb eines Strings.

Output: logische Variable
True, wenn enthalten
False, wenn nicht enthalten

Funktioniert auch mit einer Liste an patterns, die nacheinander geprüft werden → daher **[ ]**

Da einige Sonderzeichen spezielle Funktionen in Python besitzen (z.B. * als Mal-Zeichen) müssen sie ins raw string-Format übersetzte weren → daher **r" "**

Code Strukturen



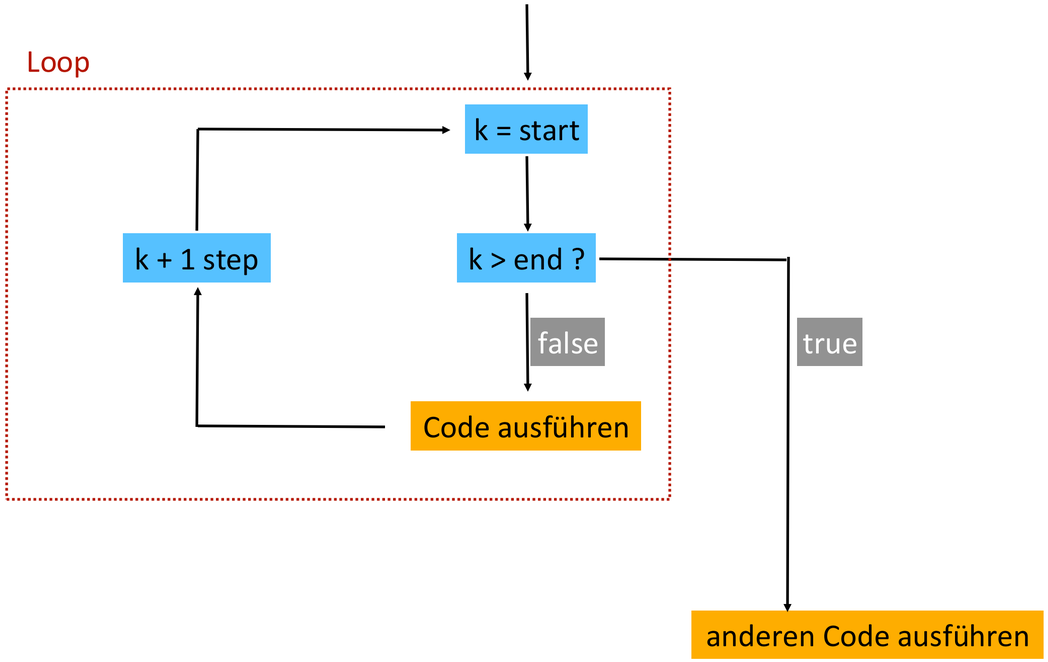
Schleifen

- In psychologischen Experimenten wird häufig mit großen Stichproben gearbeitet.

- Beispiel: Experiment, an dem 100 VPs teilnehmen und auf unterschiedliche Stimuli reagieren müssen. Das Experiment beinhaltet 80 Trials.
  - Reaktionszeiten müssen für jede VP für jeden Trial einem Stimulus zugeordnet werden…
  - 100 mal…80…macht 800 Mal ein und dieselbe Aktion?!

- Lösung: Loops!

Schleifen



- k ist ein Index, der sich mit jeder Interaktion erhöht.
- Die Schleife wird so oft wiederholt, bis k eine vorab gesetzte Grenze erreicht.

Achtung! Wenn keine (sinnvolle) Grenze gesetzt ist, bleibt man in einer Endlosschleife hängen.

For-Loop

- einen Vorgang wiederholen oder etwas wiederholt prüfen und es ist bereits vorher bekannt, wie oft er wiederholt werden soll

  - bei Dictionaries nur Keys?

```
my_list = [5, 4, 3, 2, 0, 9, 100]

# for loop checking a list
for i in range(len(my_list)):

    if(my_list[i] > 0):

        print("Good one!")
```

nach jedem Loop
i+1 bis i= 7

i als Index
—> i soll my_list elemtweise durchgehen
—> len(my_list) = 7 —> Loop wiederholt sich 7 Mal
—> range(len(my_list)) → [0 1 2 3 4 5 6]*

In jedem Loop wird dieser Code ausgeführt

* range() kann auch mit einem Start- und Endpunkt verwendet werden: range(0,7) —> [0 1 2 3 4 5 6]

List comprehension

- Statt eine Liste über ihre Indizes durchzugehen, kann man auch über die einzelnen Elemente selbst iterieren!

```
my_list = [5, 4, 3, 2, 0, 9, 100]

# for loop checking a list
for value in my_list:

    if(value > 0):

        print("Good one!")
```

- Außerdem kann man sich bei Datenstrukturen, deren Charakteristika zunutze machen

```
# looping tuples
grade_list = [("Mathhew", "A"),
              ("Lucy", "A"),
              ("Keaton", "C"),
              ("Juliet", "C")]

for name, grade in grade_list:
    if(grade == "A"):
        print(f"Excellent job, {name}!")
```

```
# looping lists in lists
animal_list = [["Golen Retriever", "Labrador"], # dogs
               ["Maine Coon", "American Shorthair"], # cats
               ["Belgian Hare", "Minilop"]] # rabbits

for animal in animal_list:
    for breed in animal:
        print(f"Do you have a {breed} at home?")
```

Recap: If/Match-Statement & Loops

```
### Zusatz: Prüft auch auf mindestens eine Zahl und ein Sonderzeichen
password = input("Bitte geben Sie ein Passwort ein: ")

# Variablen zur Überprüfung der Kriterien
has_upper = any(c.isupper() for c in password)  # mindestens ein Großbuchstabe
has_digit = any(c.isdigit() for c in password)  # mindestens eine Zahl
has_special = any(c in '!@#$%^&*(),.?":{}|<>' for c in password)  # mindestens ein
Sonderzeichen
length_ok = len(password) >= 8  # mindestens 8 Zeichen

# Überprüfung der Passwortstärke mit match
match (length_ok, has_upper, has_digit, has_special):
    case (True, True, True, True):
        print("Das Passwort ist stark.")
    case _:
        print("Das Passwort ist schwach. Bitte verwenden Sie mindestens 8 Zeichen,
einschließlich Großbuchstaben, Zahlen und Sonderzeichen.")
```

While-Loop

- einen Vorgang wiederholen oder etwas wiederholt prüfen ohne im Voraus zu wissen, wie oft es wiederholt werden soll
- innerhalb der Schleife wird geprüft, ob angehalten werden soll

```
my_list = [5, 4, 3, 2, 0, 9, 100]

i = 0
while i < len(my_list) - 1:
    i += 1
    print(f"Value at {i} is {my_list[i]}")
```

i als „Counter“
—> i soll kleiner als my_list bleiben
—> len(my_list) = 7 —> Loop wiederholt sich 7 Mal

Counter erhöhen

nach jedem Loop
i+1 bis i≥ 7

In jedem Loop wird dieser Code ausgeführt

f-strings (Formatted String Literals)

- Um Variablen in einen Satz eingefügt auszugeben, gibt es mehrere Möglichkeiten

```
my_number = 20

# Konkatenierung:
print("My number is: " + str(my_number))

# f-string
print(f"My number is: {my_number}")
```

- Auch mitten im Satz möglich:

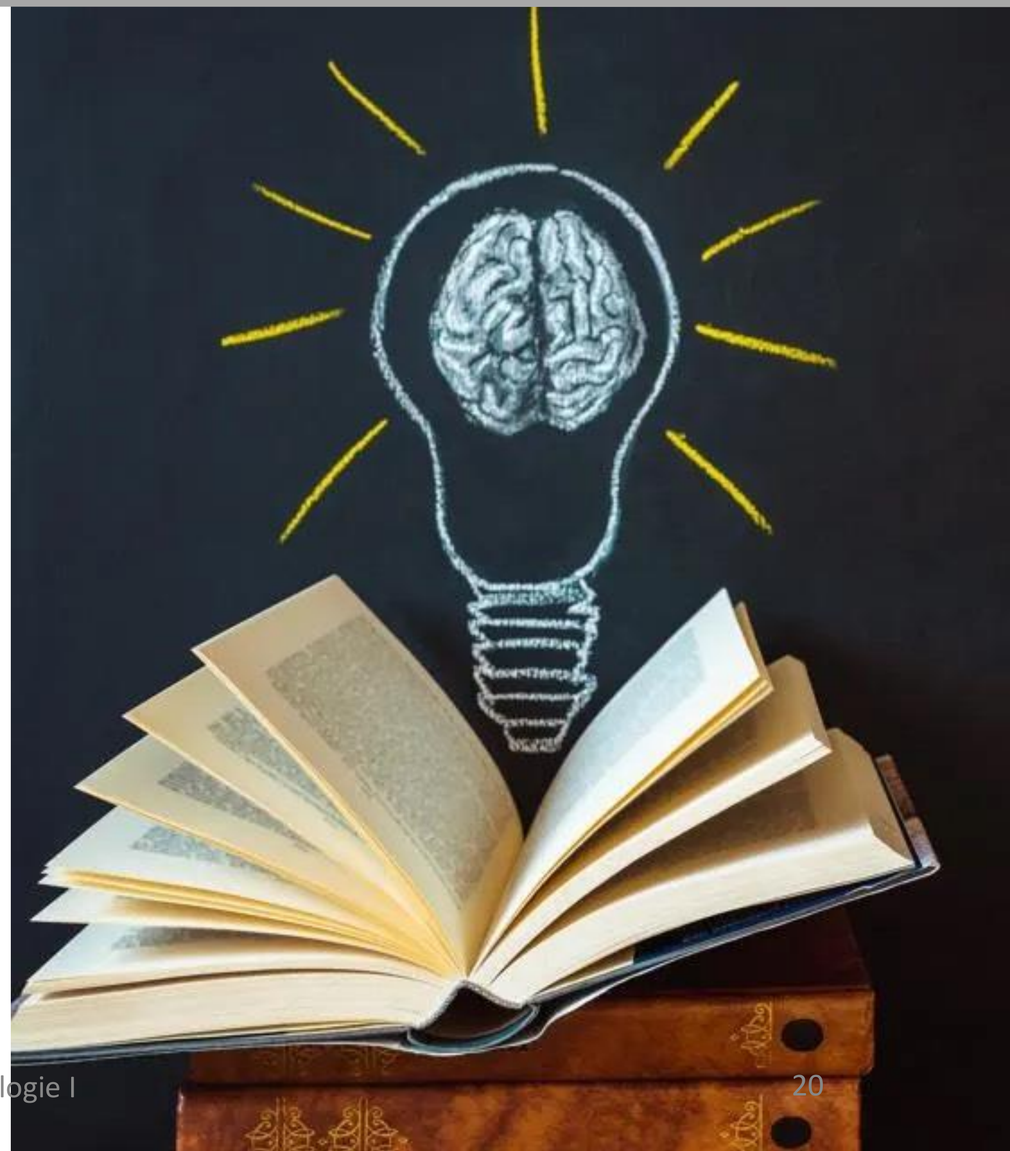
```
# Konkatenierung:
print("Number " + str(my_number) + " is correct.")

# f-string
print(f"Number {my_number} is correct.")
```

- Zeilenumbrüche werden mit `\n` (newline) eingefügt

Textfiles

- In psychologischen Experimenten werden oft Verhaltensdaten aufgezeichnet und in Textfiles gespeichert, z.B. Reaktionszeiten.

- In Python gibt es Funktionen, mit denen Files eines bestimmten Formats eingelesen werden können.
  - read, readlines
  - split
  - write, writelines



Text einlesen – read() & readlines()

- **read()**
  - liest den gesamten Inhalt der Datei als einen string
  - ineffizient bei sehr großen Files
  - Output: single string, der den gesamten Inhalt der Datei enthält, incl. Zeilenumbruchzeichen (\n)

```
# using read()
with open('example.txt', 'r') as file:
    content = file.read()

print(content)

# Output
First line
Second line
Third line
```

- **readlines()**
  - liest alle Zeilen der Datei und gibt sie als Liste zurück, wobei jede Zeile ein Element der Liste ist
  - Output: Liste von strings, einschließlich Zeilenumbruch (\n) am Ende jeder Zeile

```
# using readlines()
with open('example.txt', 'r') as file:
    lines = file.readlines()

print(lines)

# Output
['First line\n', 'Second line\n', 'Third line\n']
```

Text schreiben – write() & writelines()

- **write()**
  - schreibt einen single string in ein Textfile
  - Output wird im Schreibmodus ('w') geöffnet, wodurch die Datei erstellt wird oder überschrieben wird
  - \n zum Einfügen neuer Zeilen

```python
# using write()
with open('output.txt', 'w') as file:
    file.write("This is the first line.\n")
    file.write("This is the second line.\n")
```

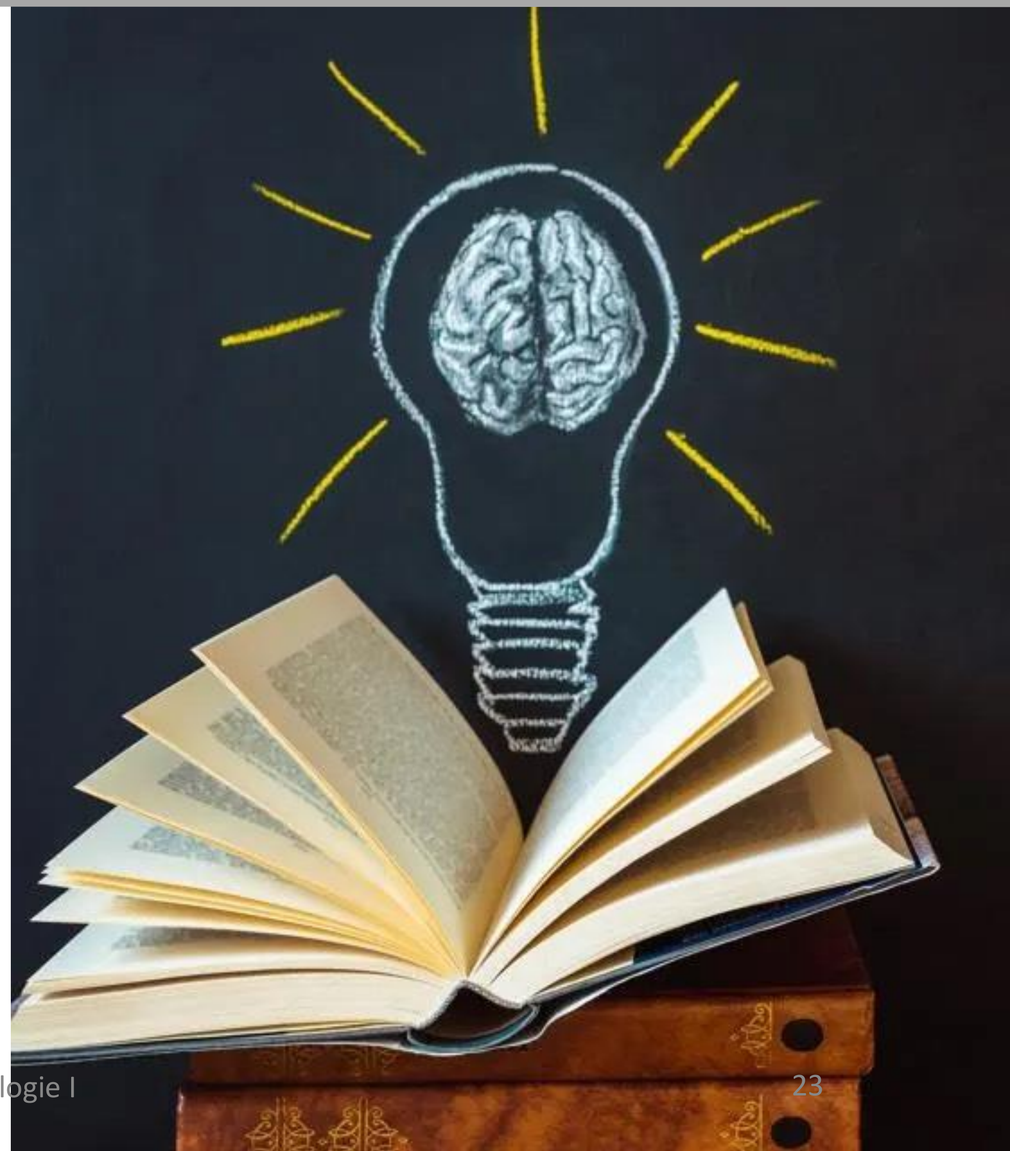
- **writelines()**
  - nimmt eine Liste von Zeichenketten und schreibt sie in die Datei
  - Zeilenumbruchzeichen (\n) werden nicht automatisch hinzugefügt

```python
# using writelines()
lines = ["First line\n", "Second line\n", "Third line\n"]

with open('output.txt', 'w') as file:
    file.writelines(lines)
```

Text schreiben – append mode

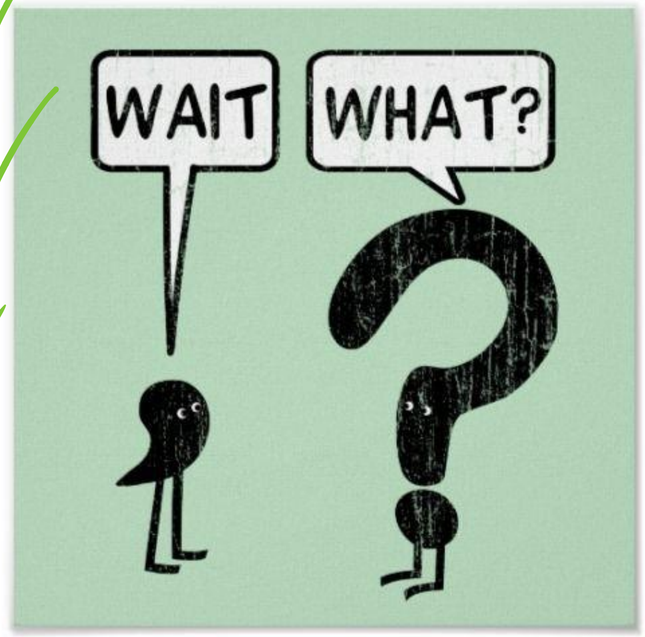
- Anfügemodus ('a') öffnen → einer bestehenden Datei Inhalte hinzufügen, ohne sie zu überschreiben
- neuer Inhalt wird am Ende der Datei hinzugefügt, ohne den vorhandenen Inhalt zu löschen

```
# append mode
with open('output.txt', 'a') as file:
    file.write("This line is appended.\n")
```



Your turn!

- For-Loops:
  - **Summe der Zahlen:** Schreibe einen Loop, der die Summe der Zahlen von 1 bis n (Input-Variable) berechnet und ausgibt.
  - **Vokalzählung:** Zähle die Anzahl der Vokale (a, e, i, o, u) in einem Input-String.

- While-Loops:
  - **Stop-Eingabe:** Schreibe einen Loop, der Benutzereingaben in einer Schleife sammelt, bis der Benutzer "stop" eingibt. Das Programm gibt dann alle gesammelten Eingaben aus.
  - **Zufallszahlen raten:** Schreibe einen Loop, der eine Zufallszahl zwischen 1 und 100 wählt und den Benutzer dazu auffordert, sie zu erraten. Das Programm gibt Hinweise, ob die Schätzung zu hoch oder zu niedrig ist, und fährt fort, bis die Zahl richtig erraten wurde. (Tipp: Vielleicht ist hier das Package „random“ hilfreich.)

- Textfiles: Bei StudIP ist ein Textfile hochgeladen (04_read_in_example.txt), dass ihr einlesen könnt und ein eigenes Textfile als als Antwort schreiben und hochladen könnt.